

THE BRITISH COLLEGE

School of Computing

BSc (Hons) Artificial Intelligence – Level 5

Group Project (100% Assessment)

GROUP PROJECT TITLE APPROVAL

1. Group Information

Group Number: 5

Group Members

Student Name	Student ID	Intake	Contact Email
Simona Kattel	24030159	September	ksimona24@tbc.edu.np
Pradip Yadav	24036544	September	ypradip24@tbc.edu.np
Sachin Maharjan	24036543	September	msachin24@tbc.edu.np
Aayushma Dhakal	24030146	September	daayushma24@tbc.edu.np
Abhinav Sitaula	23085179	September	sabhinav23@tbc.edu.np
Bardan Thapa	23085198	September	tbardan23@tbc.edu.np

(Maximum group size: 6 students as per module guidelines.)

2. Proposed Project Title

“Enterprise Retail & Strategic Inventory System”

3. Project Background & Problem Statement

In **December 2013** a famous large scale retail store called Target suffered a huge POS related data breach, hackers injected malware in their POS system compromising 40 million credit cards and 70 million personal data. A 2026 retail survey and industry reports found out that supermarkets continue to suffer the loss of millions due to hidden costs of billing systems, not just the fees, but also revenue loss during system downtime and transaction failures. And in the same report it was reported that stores often complain about their software being relatively slow for the price they are paying. Problems like this should not be ignored because it not only includes sensitive data of the users but also makes users lose their trust on systems like this. For a country like Nepal where more than 40% of the population either owns a business or are employed in this sector, a problem like this becomes extremely significant. Users of this billing software are store owner, manager, cashiers, customers, inventory manager, accountant, suppliers, and stakeholders are business owners, sponsors, government authorities, IT team, Board of Directors, Software Vendors, Payment Gateway Providers, Franchise Owners. Mostly in current System problems like lagging

of system, lack of data security, system downtime, and expensiveness is observed. Therefore, our group chose this topic to analyse the real-world challenges and risks related with retail billing systems and understand how they have an impact on businesses and customer trust in growing markets like Nepal.

4. Project Aim

The aim of this project is to design, develop, and evaluate a cost effective, fast and secured billing system.

5. Project Objectives

1. Designing and developing a secure full-stack, cost effective, retail management system using React.js (web), React Native (mobile), and FastAPI backend.
2. Implementing an IoT-based, fast working, smart billing system using an ESP32 integrated with a 2D barcode scanner, wallet integration that transmits real-time scan data to the backend via MQTT.
3. Developing a secure authentication system using JWT and bcrypt password hashing, with Email OTP-based 2FA for stores with role-based access control within the shopkeeper dashboard.
4. Integrating real-time dashboard updates using WebSocket.
5. Implementing AI features including sales forecasting using scikit-learn.
6. Implementing a RAG-based chatbot and semantic embeddings to provide context-aware responses.
7. Containerizing the overall system using Docker and Docker Compose, as it ensures consistent deployment, simplified setup, and portable development environments.

6. Project Scope

Included in Scope:

- Web-based frontend development using [React.js](#).
- Designing using Figma and mock-ups.
- Mobile application development using React Native.
- Backend API and business development using FastAPI.
- MySQL database design with relational schema and indexing.
- JWT-based authentication from bcrypt password hashing.
- Email OTP-based 2FA for shopkeepers.
- ESP32-based IoT device integration with 2D barcode scanner.
- IoT communication using MQTT protocol.
- Real-time dashboard updates using WebSocket.
- AI and analytics feature like Sales forecasting and RAG-based chatbot.

- Docker containerization and Docker Compose deployment.
- Functional and performance testing.

Out of Scope:

- Large-scale enterprise deployment.
- Commercial cloud hosting and high-availability clustering.
- Multi-branch retail chain integration.
- Advanced financial accounting systems.
- Integration with external payment gateways.
- Production-level cybersecurity auditing.
- Large-scale distributed microservice orchestration.

7. System Architecture Overview

Our software “Enterprise Retail & Strategic Inventory System” follows a modular monolithic architecture where the FastAPI backend acts as the central orchestration layer connecting IoT devices, web and mobile applications, database services, and AI components.

Which reduces system complexity and ensures reliable integration of multiple technologies within a single development environment.

a) Client Layer

The client layer consists of two applications that communicate exclusively with the backend through secure APIs.

Web Application – Shopkeeper Dashboard (React.js)

Provides the main operational interface for shopkeepers, including:

- JWT authentication with Email OTP (2FA)
- Billing and invoice generation
- Product and inventory management
- Sales analytics visualization
- AI chatbot interaction.
- Real-time updates via WebSocket

Mobile Application (React Native - Customer)

Allows customers to:

- Register and log in securely.
- View purchase history and digital receipts.
- Monitor spending analytics.
- Interact with the AI assistant.

b) Backend Core Layer (FastAPI)

FastAPI serves as the central system gateway responsible for:

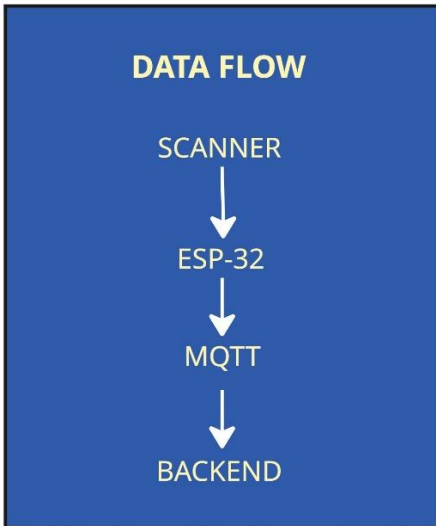
- REST API communication.
- Authentication and role-based access control
- Business logic and transaction processing
- MQTT subscription for IoT data.
- Real-time updates using WebSocket
- Communication with AI Services
- Database management and audit logging

c) Data Layer (MySQL Database)

A MySQL relational database stores system data including Users, Products, Transactions, Transaction Items, Inventory records, and Audit Logs. Database access is handled solely by the backend to ensure consistency and security.

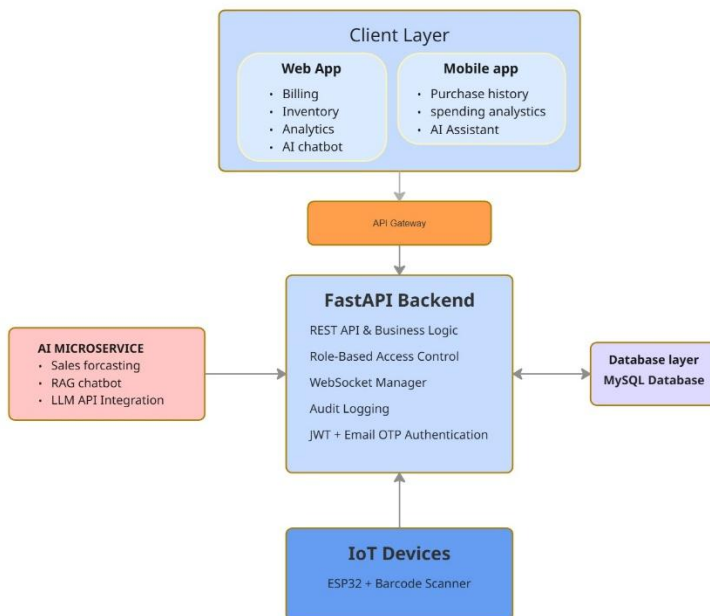
d) IoT Integration

An ESP32 microcontroller connected to a 2D barcode scanner, captures product barcodes, QR codes during billing and publishes scan data via MQTT.



e) AI Services Layer

AI functionality operates as a containerized module controlled by the backend.



System Architecture Diagram made using Miro

8. Technology Stack Declaration

Component	Selected Technology
Frontend web	React
Frontend app	React Native
Backend	FastAPI
Database	MySQL
Authentication	JWT, bcrypt, Role-based access, Email OTP
IOT Hardware	ESP32 + Barcode scanning module
Communication Protocol	REST(Frontend) + MQTT (IOT)
Real-Time communication	WebSocket
AI / Analytics Method	Scikit-learn, Vector DB, LLM API, Embedding Model
Version control	GitHub
Development Tools	VS code, Figma
Containerization	Docker, Docker Compose
Testing tools	JEST (Frontend), Pytest (Backend)

9. AI / Intelligent Component

Our software integrates two focused artificial intelligence features that enhance retail decision making and overall user experience.

The primary AI component is sales forecasting, utilizing supervised learning, especially regression techniques from the scikit-learning library. The models are trained in historical transaction data generated within the system. To evaluate model performance, standard regression metrics are used, specifically Root Mean Square Error (RMSE) and Mean Absolute Error (MAE) with a target error rate below 15%.

The secondary AI feature is a Retrieval-Augmented Generation (RAG) chatbot. It is designed to assist both customers and shopkeepers with their queries. These features use pre-trained sentence embedding models to convert store-related information into vector form.

Altogether, these features provide practical and reliable intelligence for retail decision making while remaining achievable within the project timeline.

10. IoT Component

- Hardware device to be Used: ESP32 microcontroller integrated with a 2D barcode scanner module for reading both 1D and QR barcodes.
- Sensors Integrated: Optical barcode scanning sensor. The scanner captures encoded product data during the billing process.

- Data transmission mechanism: When a product is scanned, the ESP32 reads the barcode and publishes the data via MQTT to the backend using Wi-Fi. Data flow: Scanner → ESP32 → MQTT Publish → Backend → Billing & Inventory Update.
- Security/authentication of device: The device uses a dedicated MQTT username and password, along with a static device ID included in the payload. Optional TLS can be implemented for encrypted communication.
- Expected type of data collected: Product barcode, Device ID, Timestamp of scan.

11. Advanced / Out-of-Syllabus Component

- Docker Deployment
- MQTT Protocol
- Real-time Dashboard (WebSocket)
- Email Notification
- Python Microservice for AI
- RAG Based Chatbot (LLM API Based)

Brief explanation:

The system also includes a few advanced technical components that are not part of the syllabus to enhance intelligence, efficiency, and effectiveness of system deployment. The system utilizes the MQTT protocol for efficient real-time communication between the ESP32 barcode scanner and the backend system, which facilitates rapid billing system updates. A WebSocket-based real-time dashboard also enables shopkeepers to receive instant updates on transactions and inventory. The application is containerized using Docker, which facilitates efficient system deployment and setup. A Python microservice is also provided for AI-related tasks such as sales forecasting and RAG-based chatbot development using semantic embeddings and FAISS. Email notification also supports secure OTP-based authentication for shopkeepers.